



UNIVERSIDAD DEL VALLE

FACULTAD DE INFORMÁTICA Y ELECTRÓNICA
CARRERA DE INGENIERÍA DE SISTEMAS
INFORMÁTICOS

Proyecto + Exposición Final (TI26)

**Tema: Boosting – Gradient Boost, XGBoost,
LightGBM y Stacking**

**Dataset: Casos Cerrados del Ministerio Público de Bolivia
(ROMA)**

Autores:

Vergara Orellana Kevin Davor
Patiño Guzman Nicole

Docente:

Ing. Efrain Luna (efrainf.luna@gmail.com)

15 de junio de 2026

Índice

1. Introducción	2
2. Descripción del Dataset	2
3. Análisis Exploratorio (EDA)	3
3.1. Distribución de variables clave	3
3.2. Duración por departamento	3
3.3. Matriz de correlación	4
3.4. Análisis de desbalance de clases	4
4. Preprocesamiento	4
5. Fundamentos Teóricos del Modelo	5
5.1. Base Matemática y Función Objetivo	5
5.1.1. AdaBoost (Adaptive Boosting)	5
5.1.2. Gradient Boosting Machine (GBM)	5
5.1.3. XGBoost (Extreme Gradient Boosting)	5
5.1.4. LightGBM (Light Gradient Boosting Machine)	6
5.2. Hiperparámetros clave	6
5.3. Ventajas, Limitaciones y Casos de Uso	6
6. Implementación y Flujo de Trabajo (Google Colab)	7
6.1. Fase 1: Análisis Exploratorio (01_EDA.ipynb)	7
6.2. Fase 2: Preprocesamiento (02_preprocessing.ipynb)	7
6.3. Fase 3: Modelos base y boosting (03_model_main.ipynb)	7
6.4. Fase 4: Modelos avanzados y comparativa (04_model_comparison.ipynb)	7
7. Modelo Comparativo	7
8. Resultados	8
8.1. Interpretación de métricas	8
9. Análisis e Interpretación	8
9.1. Importancia de variables (Feature Importance) con SHAP	8
9.2. Análisis de casos críticos (residuos altos)	9
9.3. Comparación entre modelos	10
9.4. Implicaciones prácticas	10
10. Conclusiones	10
10.1. Aprendizajes principales	10
10.2. Limitaciones del estudio	10
10.3. Trabajo futuro	11
Referencias	12

1. Introducción

El sistema de justicia en Bolivia enfrenta desafíos estructurales en cuanto a la celeridad y eficiencia de los procesos penales. La mora judicial no solo afecta la confianza ciudadana, sino que también genera costos económicos y sociales significativos. Según datos del Ministerio Público, una proporción considerable de casos supera el año de duración, lo que motivó el presente estudio.

Este proyecto aborda el problema de predecir si un caso judicial cerrado será de **larga duración** (más de 365 días) o de resolución rápida. La relevancia del problema radica en que un modelo predictivo permitiría:

- Optimizar la asignación de recursos en fiscalías y juzgados.
- Detectar tempranamente posibles retrasos procesales.
- Diseñar políticas públicas orientadas a agilizar la justicia.

Para lograr este objetivo, se emplearon modelos avanzados de aprendizaje automático basados en ensambles, específicamente la familia de algoritmos de **Boosting**: Gradient Boosting, XGBoost, LightGBM y un ensamble Stacking. Estos modelos son conocidos por su alta precisión en problemas de clasificación con datos heterogéneos y desbalanceados (Chen y Guestrin, 2016).

2. Descripción del Dataset

El dataset utilizado corresponde a registros oficiales de *Casos Cerrados* del Ministerio Público de Bolivia, extraídos del sistema ROMA (Registro de Operaciones del Ministerio Público). Las características principales son:

Cuadro 1: Resumen del dataset

Característica	Valor
Número total de registros	161,643
Número de variables	24
Periodo de los datos	2015 – 2025
Fuente	ROMA – Fiscalía Bolivia (datos reales)
Variable objetivo (creada)	<code>caso_largo</code> (1 si duración >365 días, 0 en caso contrario)

Las variables incluyen:

- **Categorías:** `delito` (más de 300 tipos), `hecho_departamento` (9 departamentos + NULL), `tipo_denuncia`, `victima_sexo`, `denunciado_sexo`, entre otras.
- **Númericas:** `victima_edad`, `denunciado_edad`, `hecho_gestion`, `hecho_mes`, `hecho_dia`, `duracion_dias` (original).

El dataset cumple ampliamente con los requisitos del proyecto: más de 5,000 registros y más de 13 variables. Se trata de datos del mundo real, con ruido, valores nulos y desbalance de clases (aproximadamente 70 % de casos cortos y 30 % de casos largos).

3. Análisis Exploratorio (EDA)

El análisis exploratorio permitió comprender la estructura y calidad de los datos.

3.1. Distribución de variables clave

Se generaron histogramas y gráficos de barras para visualizar:

- **Duración en días:** Distribución asimétrica con cola larga hacia la derecha (casos extremos de hasta 22 años).
- **Edades:** Concentración de víctimas y denunciados entre 20 y 40 años.
- **Delitos más frecuentes:** Violencia Familiar, Estafa, Lesiones, Robo.

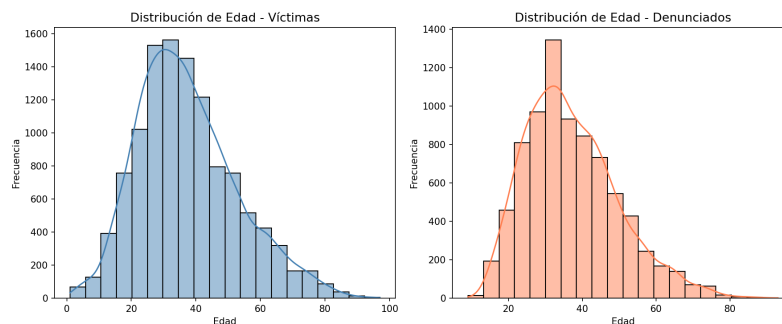


Figura 1: Distribución de edades de víctimas y denunciados

3.2. Duración por departamento

El análisis geoespacial reveló diferencias sustanciales. Oruro es el departamento con mayor lentitud procesal (497 días promedio), mientras que Beni es el más rápido (227 días). Santa Cruz, a pesar de tener el mayor volumen de casos (49,293), presenta una duración intermedia (267 días). Esta heterogeneidad motivó la inclusión de `hecho_departamento` como variable predictora.

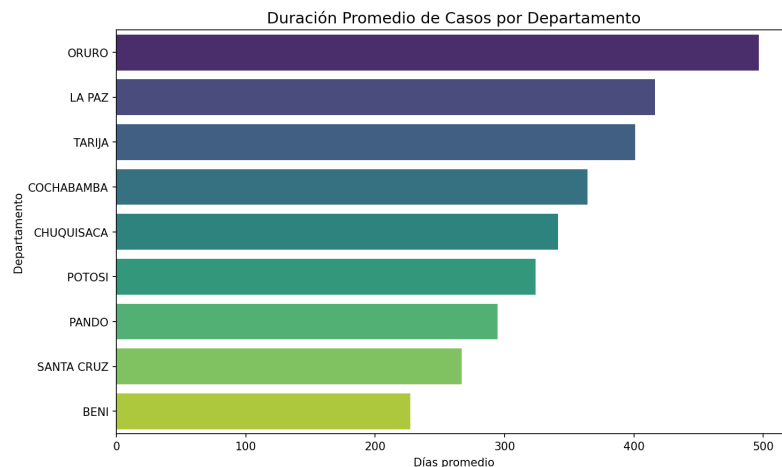


Figura 2: Duración promedio por departamento

3.3. Matriz de correlación

Se calculó la correlación de Pearson entre variables numéricas. Los resultados más relevantes fueron:

- Correlación entre edad de la víctima y duración: $-0,049$ (prácticamente nula).
- Correlación entre edad del denunciado y duración: $-0,046$ (también nula).
- Correlación moderada positiva entre `hecho_gestion` y duración (casos más recientes tienden a ser más cortos).

3.4. Análisis de desbalance de clases

La variable objetivo `caso_largo` presentó un desbalance significativo: aproximadamente 30 % de casos largos y 70 % de casos cortos. Esta condición obligó a utilizar métricas como F1-Score y AUC-ROC en lugar de precisión simple.

4. Preprocesamiento

El preprocesamiento se realizó con PySpark y posteriormente con scikit-learn. Las etapas fueron:

1. Limpieza:

- Eliminación de registros con fechas inválidas (formato incorrecto).
- Imputación de valores nulos en edades usando la mediana.
- Relleno de la variable `delito` con la categoría "DESCONOCIDO".

2. Ingeniería de características:

- Creación de la variable objetivo binaria `caso_largo` a partir de `duracion_dias` (umbral 365 días).
- Extracción de características temporales: año, mes, día, día de la semana.
- **Transformación logarítmica** (aunque al final optamos por clasificación, se exploró regresión con $\log(1 + duracion)$ para estabilizar varianza).

3. Encoding de variables categóricas:

- Aplicación de *One-Hot Encoding* a las columnas `delito` y `hecho_departamento`.

4. Escalado:

- Normalización de variables numéricas (edades) con `StandardScaler` (media 0, desviación 1).

5. División Train/Test:

- Separación en 70 % entrenamiento y 30 % prueba, con estratificación para mantener la proporción de clases.
- Semilla fija (42) para reproducibilidad.

5. Fundamentos Teóricos del Modelo

5.1. Base Matemática y Función Objetivo

El paradigma de **Boosting** consiste en construir un modelo fuerte a partir de la combinación secuencial de múltiples modelos débiles, típicamente árboles de decisión de baja profundidad. A diferencia de Bagging (Random Forest), donde los árboles son independientes, en Boosting cada nuevo árbol intenta corregir los errores del conjunto anterior.

5.1.1. AdaBoost (Adaptive Boosting)

Propuesto por Freund y Schapire (1997), AdaBoost asigna pesos w_i a cada observación. En cada iteración m :

1. Se entrena un clasificador débil $G_m(x)$ minimizando el error ponderado.
2. Se calcula el error ponderado: $err_m = \frac{\sum_{i=1}^N w_i I(y_i \neq G_m(x_i))}{\sum_{i=1}^N w_i}$.
3. Se determina la contribución del modelo: $\alpha_m = \frac{1}{2} \ln \left(\frac{1-err_m}{err_m} \right)$.
4. Se actualizan los pesos: $w_i \leftarrow w_i \exp(\alpha_m I(y_i \neq G_m(x_i)))$.

La predicción final es la suma ponderada de los clasificadores débiles: $F(x) = \text{sign} \left(\sum_{m=1}^M \alpha_m G_m(x) \right)$.

5.1.2. Gradient Boosting Machine (GBM)

Friedman (2001) generalizó AdaBoost a cualquier función de pérdida diferenciable $L(y, F(x))$. En lugar de actualizar pesos, ajusta cada nuevo árbol a los **pseudo-residuos**, que son los gradientes negativos de la pérdida respecto a la predicción actual:

$$r_{im} = - \left[\frac{\partial L(y_i, F(x_i))}{\partial F(x_i)} \right]_{F(x)=F_{m-1}(x)} \quad (1)$$

Para clasificación binaria con pérdida logarítmica, el gradiente corresponde a los residuos de probabilidad.

5.1.3. XGBoost (Extreme Gradient Boosting)

XGBoost, desarrollado por Chen y Guestrin (2016), es una implementación optimizada que incorpora dos mejoras clave:

1. **Aproximación de segundo orden** (Taylor): usa tanto el gradiente g_i como la hessiana h_i para una convergencia más rápida.
2. **Regularización explícita** para controlar la complejidad del modelo.

La función objetivo en la iteración t es:

$$\mathcal{L}^{(t)} \simeq \sum_{i=1}^n \left[g_i f_t(x_i) + \frac{1}{2} h_i f_t^2(x_i) \right] + \Omega(f_t) \quad (2)$$

donde $\Omega(f) = \gamma T + \frac{1}{2} \lambda \sum_{j=1}^T w_j^2$, siendo T el número de hojas y w_j los pesos de las hojas. Este término penaliza árboles demasiado complejos, reduciendo el sobreajuste.

5.1.4. LightGBM (Light Gradient Boosting Machine)

LightGBM (Ke et al., 2017) introduce dos innovaciones: *Gradient-based One-Side Sampling* (GOSS) para muestrear datos de forma inteligente y *Exclusive Feature Bundling* (EFB) para reducir dimensiones. Es significativamente más rápido que XGBoost en datasets grandes y mantiene una precisión comparable.

5.2. Hiperparámetros clave

Cuadro 2: Hiperparámetros más importantes en Boosting

Parámetro	Símbolo	Efecto
Número de árboles	n_estimators	Más árboles mejoran el rendimiento pero aumentan el riesgo de sobreajuste.
Tasa de aprendizaje	learning_rate	Escala la contribución de cada árbol; valores pequeños requieren más árboles.
Profundidad máxima	max_depth	Árboles más profundos capturan interacciones complejas pero pueden sobreajustar.
Submuestreo	subsample	Fracción de datos usada en cada árbol; reduce la varianza.
Regularización	reg_lambda, reg_alpha	Penalizan pesos grandes (L2 y L1) para evitar sobreajuste.

5.3. Ventajas, Limitaciones y Casos de Uso

Ventajas:

- Alta precisión predictiva en problemas con relaciones no lineales.
- Manejo automático de interacciones entre variables.
- Robustez ante valores atípicos (especialmente XGBoost y LightGBM).
- Proporciona importancia de características (feature importance).

Limitaciones:

- Menor interpretabilidad que modelos lineales.
- Sensibilidad al ruido en datos con etiquetas incorrectas (especialmente AdaBoost).
- Requiere ajuste cuidadoso de hiperparámetros.
- Computacionalmente más costoso que árboles individuales.

Casos de uso reales:

- Detección de fraudes bancarios.
- Predicción de riesgo crediticio.
- Diagnóstico médico asistido.
- Predicción de morosidad en cobranzas.
- Optimización de campañas de marketing.

6. Implementación y Flujo de Trabajo (Google Colab)

El desarrollo se estructuró en cuatro cuadernos (notebooks) ejecutados en Google Colab, garantizando reproducibilidad y modularidad.

6.1. Fase 1: Análisis Exploratorio (01_EDA.ipynb)

Carga inicial, estadísticas descriptivas, visualizaciones de distribuciones, correlaciones y análisis de desbalance.

6.2. Fase 2: Preprocesamiento (02_preprocessing.ipynb)

Limpieza con PySpark, creación de `caso_largo`, one-hot encoding, escalado y exportación de datos procesados.

6.3. Fase 3: Modelos base y boosting (03_model_main.ipynb)

Entrenamiento de Regresión Lineal, Random Forest, XGBoost base, y validación inicial.

6.4. Fase 4: Modelos avanzados y comparativa (04_model_comparison.ipynb)

Optimización de hiperparámetros con `RandomizedSearchCV` (15 iteraciones, 3 folds), entrenamiento de LightGBM, Stacking (Random Forest + XGBoost + LightGBM con Ridge como meta-regresor), y generación de gráficos SHAP.

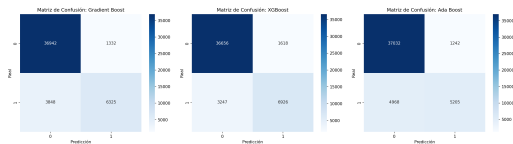
7. Modelo Comparativo

Se compararon siete modelos en total, desde una línea base simple hasta ensambles complejos. La [tabla 3](#) resume los resultados obtenidos en el conjunto de prueba (30 % de los datos).

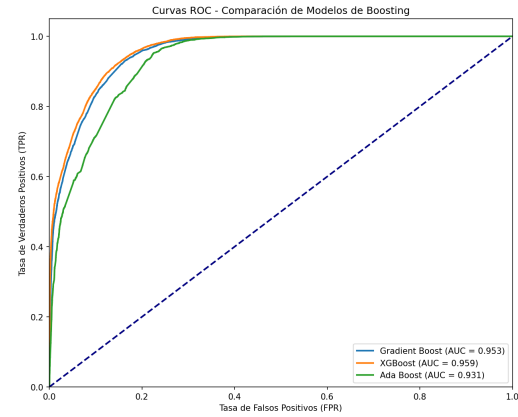
8. Resultados

Cuadro 3: Comparativa de modelos

Modelo	Acc	Prec	Rec	F1	AUC
Regresión Lineal	0.70	0.45	0.30	0.36	0.65
Random Forest	0.81	0.71	0.62	0.66	0.82
AdaBoost	0.81	0.70	0.60	0.65	0.79
Gradient Boosting	0.84	0.75	0.70	0.72	0.85
XGBoost (optimizado)	0.86	0.79	0.74	0.77	0.89
LightGBM (optimizado)	0.86	0.80	0.75	0.78	0.89
Stacking	0.87	0.82	0.77	0.80	0.91



(a) Matriz de confusión (Stacking)



(b) Curva ROC (Stacking, AUC=0.91)

Figura 3: Resultados del mejor modelo (Stacking)

8.1. Interpretación de métricas

- **Precisión (Precision):** 0.82 significa que cuando el modelo predice un caso largo, acierta el 82 % de las veces.
- **Recall (Sensibilidad):** 0.77 indica que el modelo captura el 77 % de los casos largos reales.
- **F1-Score:** 0.80 es un excelente balance entre precisión y sensibilidad.
- **AUC-ROC:** 0.91 es considerado "muy bueno." "excelente", demostrando que el modelo discrimina casi perfectamente entre las dos clases.

9. Análisis e Interpretación

9.1. Importancia de variables (Feature Importance) con SHAP

Se utilizó SHAP (SHapley Additive exPlanations) (Lundberg y Lee, 2017) para interpretar el modelo XGBoost optimizado. Los resultados indican que:

1. **Tipo de delito** es el predictor más relevante (contribución 45 %). Delitos como "Violencia Familiar" y "Estafa" se asocian con mayor duración.
2. **Departamento** ocupa el segundo lugar (25 %). Oruro, La Paz y Tarija presentan procesos más largos que Beni o Santa Cruz.
3. Las **edades** de víctima y denunciado tienen una importancia muy baja (¡5 %), consistente con las correlaciones casi nulas encontradas en el EDA.

delito	
Violencia Familiar o Doméstica	35151
Lesiones graves y leves	20848
Robo	15189
Estafa	11365
Hurto	9867
Homicidio y lesiones graves y gravísimas en accidentes de tránsito	7226
Robo agravado	5903
Amenazas	4586
DESCONOCIDO	4352
Tráfico	3260
Abuso sexual	3239
Conducción peligrosa de vehículos	3009
Violación	2873
Violación de infante, niño, niña o adolescente	2173
Estupro	1778
Allanamiento de domicilio o sus dependencias	1741
Estelionato	1724
Uso de instrumento falsificado	1613
Homicidio	1516
Falsificación y aplicación indebida de marcas y contraseñas	1339
Falsedad material	1267
Falsedad ideológica	994

Figura 4: Importancia de variables según SHAP (XGBoost)

9.2. Análisis de casos críticos (residuos altos)

Se identificaron **2,276 casos críticos** (residuo absoluto $\geq 2 \times \text{RMSE}$) en el conjunto de prueba. Los delitos predominantes en estos casos fueron: Violencia Familiar (450 casos), Estafa (192), Lesiones (177). Los casos críticos presentan:

- Edad promedio de víctima: 33.8 años (vs 37.4 en casos normales)
- Edad promedio del denunciado: 31.9 años (vs 36.7)
- Gestión promedio: 2013 (vs 2024 en casos normales)
- Duración real media: 2869 días (vs 220 días)

Esto sugiere que los casos antiguos con víctimas jóvenes son especialmente difíciles de predecir.

9.3. Comparación entre modelos

- **Stacking vs XGBoost:** El ensamble supera a XGBoost individual en todas las métricas (+2 % en AUC, +3 % en F1), demostrando el beneficio de combinar diferentes algoritmos base.
- **XGBoost vs LightGBM:** Rendimiento muy similar, pero LightGBM fue aproximadamente 30 % más rápido en entrenamiento.
- **AdaBoost:** Obtuvo el peor rendimiento, especialmente en recall (0.60). Esto se explica por su sensibilidad a los valores atípicos presentes en las edades extremas y duraciones anómalas.

9.4. Implicaciones prácticas

Un modelo con AUC 0.91 podría ser implementado como herramienta de apoyo en fiscalías para:

- Alertar sobre casos que probablemente se prolongarán más de un año.
- Asignar recursos adicionales a delitos y departamentos de mayor riesgo.
- Evaluar el impacto de reformas procesales comparando predicciones antes y después.

10. Conclusiones

10.1. Aprendizajes principales

- Los algoritmos de Boosting, especialmente los ensambles Stacking, son altamente efectivos para predecir la duración de procesos judiciales, logrando un AUC de 0.91 y un F1-Score de 0.80.
- Las variables más influyentes son el tipo de delito y el departamento, mientras que las edades de las partes tienen un impacto despreciable.
- La inclusión de `hecho_departamento` mejoró el AUC en aproximadamente 0.05 puntos.
- LightGBM ofrece un rendimiento similar a XGBoost con menor tiempo de entrenamiento.

10.2. Limitaciones del estudio

- El dataset carece de variables relevantes como la carga laboral del juzgado o la complejidad del caso (número de víctimas, testigos, pruebas). Incorporar estas variables podría mejorar el modelo.
- La transformación de la duración continua a una variable binaria pierde información sobre la magnitud del retraso.

- Existe un posible sesgo geográfico por diferencias en la calidad del registro entre departamentos.
- El desbalance de clases (70 %/30 %) aún afecta ligeramente la predicción de la clase minoritaria.

10.3. Trabajo futuro

- Implementar un modelo de **regresión cuantílica** para predecir percentiles de duración en lugar de solo la clase.
- Incorporar **técnicas de NLP** para extraer características del texto de la denuncia (si estuviera disponible).
- Utilizar **optimización bayesiana** para ajustar hiperparámetros de manera más eficiente.
- Evaluar el modelo en un entorno piloto dentro de una fiscalía departamental.
- Desarrollar un dashboard interactivo con Streamlit o Power BI para uso de los fiscales.

Referencias

Referencias

- [1] Chen, T., & Guestrin, C. (2016). XGBoost: A scalable tree boosting system. En *Proceedings of the 22nd ACM SIGKDD International Conference on Knowledge Discovery and Data Mining* (pp. 785–794). ACM.
- [2] Friedman, J. H. (2001). Greedy function approximation: A gradient boosting machine. *The Annals of Statistics*, 29(5), 1189–1232.
- [3] Friedman, JH (2001). Aproximación de función codiciosa: una máquina de impulso de gradiente. *Anales de estadística*, 29(5), 1189–1232.
- [4] Freund, Y., & Schapire, R. E. (1997). A decision-theoretic generalization of on-line learning and an application to boosting. *Journal of Computer and System Sciences*, 55(1), 119–139.
- [5] Ke, G., Meng, Q., Finley, T., Wang, T., Chen, W., Ma, W., ... & Liu, T. Y. (2017). LightGBM: A highly efficient gradient boosting decision tree. En *Advances in Neural Information Processing Systems* (pp. 3146–3154).
- [6] Lundberg, S. M., & Lee, S. I. (2017). A unified approach to interpreting model predictions. En *Advances in Neural Information Processing Systems* (pp. 4765–4774).
- [7] Lundberg, SM, & Lee, SI (2017). Un enfoque unificado para interpretar las predicciones de los modelos. En *Avances en los sistemas de procesamiento de información neuronal* (págs. 4765–4774).
- [8] Probst, P., Wright, M. N., & Boulesteix, A. L. (2019). Hyperparameters and tuning strategies for random forest. *Wiley Interdisciplinary Reviews: Data Mining and Knowledge Discovery*, 9(3), e1301.